

Lecture 17

Operator Overloading

Today's Lecture Contents

- Overloading Operators
 - Binary operators overloading
 - Overloading I/O operators
 - Postfix/prefix increment operator
 - Array subscript operator
 - Assignment operator

Introduction to Operator Overloading

- **Function Overloading**

- Function overloading provides a mechanism to create and resolve function calls to multiple functions with the same name, so long as each function has a unique function prototype.
- This allows you to create variations of a function to work with different data types, without having to think up a unique name for each variant.

- **Operator Overloading**

- In C++, operators (e.g., +, -, = etc.) are implemented as functions.
- By using function overloading on the **operator functions**, you can define your own versions of the operators that work with **user-defined data types** (i.e. classes).
- Using function overloading to overload operators is called **operator overloading**.

Operator as a function

- Consider the following example,

```
int x = 2;
```

```
int y = 3;
```

```
std::cout << x + y << '\n';
```

- The compiler comes with a built-in version of the plus operator (+) for *integer* operands.
- When compiler sees the expression $x + y$, it translates it to the function call `operator+(x, y)` (where *operator+* is the name of the function).

Operator as a function

- Now consider the following:

```
double z = 2.0;
```

```
double w = 3.0;
```

```
std::cout << w + z << '\n';
```

- The compiler also comes with a **built-in version of the plus operator (+)** for *double operands*.
- Expression `w + z` becomes function call ***operator+(w, z)***, and function overloading is used to determine that the compiler should be calling the *double version* of this function instead of the integer version.

Operator as a function

- Now consider what happens if we try to add two object of a user-defined class `Complex`:

```
Complex C1;  
Complex C2;  
std::cout << C1 + C2 << '\n';
```

- Because `Complex` is a user-defined class, the compiler does not have a built-in version of the *plus operator* that it can use for `Complex` operands.
- In order to make it work like we want, we'd need to write an overloaded function to tell the compiler how the *+ operator* should work with two operands of type `Complex`.

Binary Operator Overloading

- We can make *object operation* look like *individual variable operation*, using operator functions

Complex a, b, c;

c = add(a, b);

```
class Complex {  
    double _real, _imag;  
public:  
    double getReal() {...}  
    double getImag() {...}  
};  
  
Complex add(const Complex &a, const Complex &b) {  
    double r    = a.getReal()    + b.getReal();  
    double i    = a.getImag()    + b.getimag();  
    return Complex(r, i);    }
```

- But instead of enclosing arguments in parenthesis such as above we want something natural e.g.,
c = a + b
- operator overloading is an appropriate approach to achieve this goal.

Operator Overloading Syntax

- The definition of operator + for the Complex class is nearly the same as member *function add()*
- To overload the + operator for the Complex class, $c = a + b$:
 - Use the name + in place of the function name *add*
 - Use keyword *operator* in front of the +
 - Example:
Complex *operator +* (Complex& a...)
- So the general Syntax of operator class declaration becomes:

operator@(*argument-list*)



--- operator is a function

--- @ is one of C++ operator symbols (+, -, =, etc..)

Examples:

operator+

operator-

operator*

operator/

Implementing Operator Overloading

- Two ways:
 - Implemented as member functions
 - Implemented as non-member or Friend functions
 - the operator function may need to be declared as a friend if it requires access to protected or private data
 - Compiler translates Expression *obj1@obj2* into a function call
 - *obj1.operator@(obj2)*, if this function is defined within class obj1
- OR
- *operator@(obj1,obj2)*, if this function is defined outside the class obj1(i.e., as a friend function or non-member function)

Implementing Operator Overloading

1. Defined as a member function:

- Add this instance (left operand) with the rhs operand,
- Construct a new instance containing the sum and return it by value.
- Two ways of calling an overloaded binary operator
 - `c = a + b;`
 - `c = a.operator+(b);` //although valid but not widely used.

```
class Complex {  
    double _real;  
    double _imag;  
public:  
    ...  
Complex operator +(const Complex &obj)  
{  
    double r = _real + obj._real,  
    double i = _imag + obj._imag;  
    return Complex(r, i);  
}  
    ...  
};
```

Complex
a, b, c;

c = a + b;

lhs operator rhs

Compiler translates the above statement to the below:

c = a.operator+ (b);

Implementing Operator Overloading

2. Defined as a non-member function

- Note that non-member function will need getter functions to access object's private data.

```
class Complex {  
    double _real;  
    double _imag;  
public:  
    ...  
    double getreal() {  
return _real; }  
//need access functions  
    double getimag() {  
return _imag; }  
    ...  
};
```

Complex a,b,c;

c = a+b;

Compiler translates the above statement to the below:

c = operator+ (a, b);

```
Complex & operator +(Complex &obj1, Complex &obj2) {  
    double r = obj1.getreal() + obj2.getreal(),  
    double i = obj1.getimag() + obj2.getimag();  
    return Complex(r, i);  
}
```

Implementing Operator Overloading

3. Defined as a friend function

- using friend function, class private members can be accessed directly
- No need for the getter() function

```
class Complex {  
    double _real;  
    double _imag;  
public:  
    ...  
friend Complex operator+ (Complex &, complex &);  
    ...  
};
```

Complex a,b,c;

c = a+b;



c = operator+ (a, b);

```
Complex operator +(Complex &obj1, Complex &obj2) {  
    double r = obj1._real + obj2._real;  
    double i = obj1._imag + obj2._imag;  
    return Complex(r, i);  
}
```

We can define the binary operator overloading function using one of the method:

```
int main() {  
  
    Complex c1(3.1, 4.2);  
    std::cout << c1.real << c1.imag<<std::endl; // (3.10,4.20)  
  
    Complex c2(3.1,0);  
    std::cout << c2.real << c2.imag<<std::endl; // (3.10,0.00)  
  
    Complex c3 = c1 + c2; //compiler generates: c1.operator+(c2)  
    std::cout << c3.real << c3.imag<<std::endl; // (6.20,4.20)
```

Binary operator+ overloading

- However, the above examples don't handle the following situation:

```
Complex c1;
```

```
c1 + 2.325;
```

- Compiler translates `c1+2.325` into **`c1.operator+(double)`**.
- To provide a compatible function, we have to add another **`operator+ member function`** definition as following:

```
Complex operator +(double d)
{
    double real  = _real  + d;
    double imag = _imag;
    return Complex(real, imag) ;
}
```

- Now statements like, `c1+2.325` can be safely executed.

Binary operator+ overloading

- However, now problem arises if we try to execute the following:

```
Complex c, c1;
```

```
c = 2.325 + c1;
```

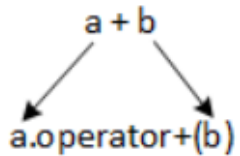
- Note that the LHS of the + operator is not an object but is a fundamental data types, i.e. int or double.
- In this case, the **only** option is to define the operator+ function as **non-member** (friend) function with two arguments:

```
class Complex {  
    ...//everything remains same  
    friend Complex operator +(double lhs, const complex &);  
    ...  
};
```

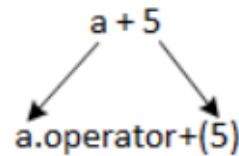
```
Complex operator +(double lhs, const complex & rhs)    {  
    double real = lhs._real    + rhs;  
    double imag = lhs._imag    +  rhs;  
    return Complex(real, imag);  
}
```

Summarizing the previous slides

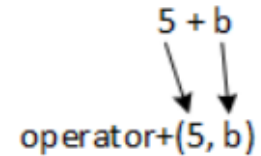
```
class Complex {  
    double _real, _imag;  
public:  
    Complex(double r=5.5, double i=2.5) : _real(r), _imag(i) {};  
    Complex operator+(Complex b);           // (a)  
    Complex operator+(double r);           // (b)  
    friend Complex operator+(double r, Complex a); // (c)  
};
```



(a)



(b)



(c)

- a. Member function where both operands are complex.
- b. Member function where the left operand is a complex and the right operand is an double.
- c. Non-member (friend) function where the left operand is an double and the right operand is a complex.

Binary operator overloading

- Other binary operators are overloaded very similar to the + operator as demonstrated in the above examples
- Examples:

```
Complex operator * (Complex & c1, Complex & c2);
```

```
Complex operator / (Complex &c1, Complex & c2);
```

```
Complex operator - (Complex & c1, Complex & c2);
```

Restrictions on Operator Overloading

- The overloaded operator must be an existing and valid operator. You cannot create your own operator such as $\oplus$.
- Certain C++ operators cannot be overloaded, such as sizeof, dot (. and .*), scope resolution (::) and conditional (?:).
- The overloaded operator must have at least one operands of the user-defined types. You cannot overload an operator working on fundamental types.
 - you can't overload the '+' operator for two ints (fundamental type) to perform subtraction. For example, `operator+(int a, int b)` will not work.
- You cannot change the syntax rules (such as associativity, precedence and number of arguments) of the overloaded operator.
 - For example, + is left associative whereas = is right associative

Restrictions on Operator Overloading

Operators that can be overloaded							
+	-	*	/	%	^	&	
~	!	=	<	>	+=	-=	*=
/=	%=	^=	&=	=	<<	>>	>>=
<<=	==	!=	<=	>=	&&		++
--	->*	,	->	[]	()	new	delete
new[]	delete[]						

Operators that cannot be overloaded				
.	.*	::	?:	sizeof

Overloading binary stream
insertion << and stream
extraction >>

Overloading inserter and extractor operator

- C++ overloads (i.e., reuses) the inserter (<<) and the extractor (>>) operators in the context of streams as the output and input operators respectively e.g.,
 - cout <<
 - cin >>
- Consider the Point class below:
- We can use overloaded insertion and extraction operator in Point class.

```
// Class Declaration
class Point {
private:
    int x, y;
public:
    Point(int x = 0, int y = 0);
    int getX() const; // Getters
    int getY() const;
    void setX(int x); // Setters
    void setY(int y);
};
```

Overloading Stream Insertion Operator (<<)

- The stream insertion operator is a *binary operator*, i.e., it takes two operands

- Usage:

```
int y=10;
```

cout << y;

LHS operator RHS

- The LHS is a C++ object of the ostream class which is in the iostream library
- The ostream class knows how to output any of the built-in C++ data types and any standard library data types
- Now consider the following statement.

```
cout<<point1; //point1 is object of Point
```

- the compiler first tries to find a member function of the ostream class (cout is an object of ostream), that accepts point object as a parameter

```
cout.operator<<(point1);
```

- However, compiler will not find operator<< in the Ostream class.
- Also since the left operand is not a Point object (cout is an ostream object), we CANNOT overload << as member function of point class, but need to use non-member function for operator overloading.

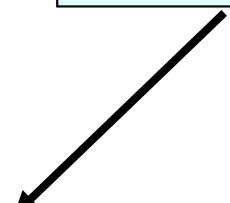
Overloading Stream Insertion Operator (<<)

- We implement operator << as a **friend function** of the Point class

- Usage

```
cout << Point;
```

cout ---- object of ostream



- Implementation

```
class Point{  
friend void operator<<( ostream& out, Point& rhs );  
//remainder of the Point class here  
};
```

friend function definition:

```
void operator<<( ostream& out, Point& rhs ){  
    out << rhs.x<<rhs.y;  
}
```

lhs (points to `ostream& out`) **rhs** (points to `Point& rhs`)

Overloading Stream Insertion Operator (<<)

- We modify our operator<< overloaded function a little.
- We return our modified ostream object by reference.
- This will enable cascaded function call, such as

```
cout << p1<<p2;
```

cout ---- object of ostream



```
class Point {  
    friend ostream& operator<<( ostream& out, Point& rhs );  
    //remainder of the Point class here  
};
```

friend function definition:

```
    ostream& operator<<( ostream& out, Point& rhs ){  
        out << rhs.x<<rhs.y;  
        return out;  
    }
```

Diagram annotations: Red arrows point from the labels 'lhs' and 'rhs' to the parameters 'ostream& out' and 'Point& rhs' respectively in the function signature.

Overloading Stream Extraction Operator (>>)

- Stream extraction **operator >>** is also implemented as a friend function
- Usage

```
cin >> Point1; //Already declared Point Point1;
```

– Implementation

cin ---- object of istream

```
class Point{  
friend istream& operator>>( istream& in, Point& rhs );  
//remainder of the Point class here  
};
```

Friend function definition:

```
istream& operator>>( istream& in, Point& rhs ){  
    cout << "Enter x and y coord: ";  
    in >> rhs.m_x >> rhs.m_y; // access private data  
    return in;  
}
```

Implementation: Overloading << and >>

```
// Class Declaration
class Point {
private:
    int x, y;
public:
    Point(int x = 0, int y = 0);
    int getX() const; // Getters
    int getY() const;
    void setX(int x); // Setters
    void setY(int y);
    //declaring operator overloading functions as friend
    friend ostream & operator<<(ostream & out, Point & point);
    friend istream & operator>>(istream & in, Point & point);
};
```

Definitions of Overloading << and >>

```
ostream & operator<<(ostream & out, const Point & point) {  
    out << "(" << point.x << "," << point.y << ")";    // access private  
data  
    return out;    //this enables cout<<a<<b<<c;  
}  
  
istream & operator>>(istream & in, Point & rhs) {  
    cout << "Enter x and y coord: ";  
    in >> rhs.x >> rhs.y;    // access private data  
    return in;    //this enables cin>>a>>b>>c;  
}
```

- Note the above overloaded >> and << functions return references in and out, which enable input and output operations, respectively, on Point objects to be **cascaded** with input and output operations on other Point objects.

Usage of Overloaded << and >> operators

```
int main() {  
    Point p1(1, 2, 0), p2;  
    // Using overloaded operator <<  
    cout << p1 << endl;  
  
    // Using overloaded operator >>  
    cin >> p1;  
    cout << p1 << endl;  
  
    cout << p1 << endl;  
    cin >> p1 >> p2;  
    cout << p1 << endl;  
    cout << p2 << endl;  
}
```

```
Point(1, 2, 0)  
Enter x and y coord: 5  
6  
Point(5, 6, 0)  
Point(5, 6, 0)  
Enter x and y coord: 9  
8  
Enter x and y coord: _
```